ارائه سر کلاس • برنامه‌نویسی اندروید

توضیح کامل فایل
activity_main.xml

طراحی ظاهر صفحه اصلی برنامه — از XML تا جاوا

XML = ظاهر صفحه

Java = منطق برنامه

پل ارتباطی: id

activity_main.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:orientation="vertical"
   android:padding="16dp"
   android:background="#F5F7FA">
   <TextView ... />
</LinearLayout>
```



۱. عنوان برنامه
۲. دکمه‌های اصلی
۳. فهرست رویدادهای ثبت‌شده

فهرست مطالب

فهرست مطالب ارائه

۱۴ بخش متن، در ۶ فصل — همراه با کد واقعی، جدول و پرسش‌وپاسخ

۰۱

آشنایی با فایل

`XML` چیست و چرا طراحی از کد جدا است؟

۰۲

ریشه `LinearLayout`

فضای نام، عرض و ارتفاع، جهت‌گیری، `padding` و رنگ پس‌زمینه

۰۳

`TextView` ها

`id` ، متن، اندازه، رنگ و `gravity`

۰۴

دکمه‌های اصلی

سه `Button` و قرارداد نام‌گذاری شناسه‌ها

۰۵

`ScrollView`

قوطی اسکرول‌شونده و ترفند `0dp + weight=1`

۰۶

جاوا و جمع‌بندی

`findViewById` و کلاس `R` ، منطق طراحی، جمع‌بندی و ۱۰ پرسش کلاس

بخش ۱

این فایل اصلاً چیست؟

این فایل یک فایل `XML` است، **نه جاوا**. اسمش `activity_main.xml` است و کارش طراحی ظاهر صفحه اصلی برنامه است.

XML چیست؟

`XML` یک زبان نشانه‌گذاری است (مثل `HTML`). کارش **توصیف ساختار** است، نه اجرای منطق.

اصل مهم اندروید

طراحی (ظاهر) را از منطق (کد) جدا کن.

فایل	کارش
`activity_main.xml`	ظاهر صفحه — دکمه کجاست؟ رنگش چیست؟
`MainActivity.java`	منطق صفحه — کلیک چه کار کند؟

بخش ۱ • برای کلاس بگو

تشبیه کلاس: نقشه ساختمان و برقکار

فایل `XML` مثل **نقشه ساختمان** است و فایل جاوا مثل **برقکار و لوله‌کش** که داخل ساختمان کار می‌کند. اول نقشه را می‌کشیم، بعد برق را وصل می‌کنیم. اگر نقشه را عوض کنیم، برقکار باید بداند کدام کلید کجاست — این ارتباط با `id` برقرار می‌شود.

این فایل کجا استفاده می‌شود؟

```
setContentView(R.layout.activity_main);
```

این خط می‌گوید: این صفحه را با نقشه `activity_main.xml` بساز.

۱ **نقشه را می‌کشیم**
XML

←

۲ **برق را وصل می‌کنیم**
Java

←

۳ **با `id` به هم وصل می‌شوند**

بخش ۲ و ۳

سرآیند فایل و کامنت‌ها

خط اول فایل

```xml
<?xml version="1.0" encoding="utf-8"?>
```

- می‌گوید: این فایل یک XML نسخه ۱.۰ است و کاراکترها با `UTF-8` کدگذاری شده‌اند.
- `UTF-8` چیست؟ استانداردی برای ذخیره حروف.
- اگر `UTF-8` نباشد، حروف فارسی ممکن است خراب نمایش داده شوند.

مثل شناسنامه فایل — می‌گوید من XML هستم و حروفم را چطور بخوان

کامنت (توضیح) بالای فایل

```
<!-- در این صفحه: صفحه اصلی اپلیکیشن رویدادهای دانشگاه
1. عنوان برنامه
2. دکمه‌های اصلی
3. فهرست رویدادهای ثبت‌شده — نمایش داده می‌شوند.
-->
```

- کامنت یعنی توضیح برای انسان — کامپیوتر آن را نادیده می‌گیرد.
- شروع با `<!--` و پایان با `-->`
- فقط برای برنامه‌نویس بعدی (یا خودت در آینده) است.

مثل یادداشت چسبان روی یخچال — کامپیوتر نمی‌خواند، برای ما مفید است

بخش ۴

ریشه: `LinearLayout` و فضای نام

LinearLayout چیست؟

یعنی **چیدمان خطی** — همه چیز را پشت سر هم بچین: یا عمودی (زیر هم) یا افقی (کنار هم).

مثل قفسه کتاب — کتاب‌ها را کنار هم یا روی هم می‌چینی

`xmlns:android` چیست؟

`XML Namespace` یعنی **فضای نام**. می‌گوید: همه چیزهایی که با `android:` شروع می‌شوند، از کتابخانه اندروید هستند.

چرا لازم است؟ چون XML عمومی است و ممکن است چند کتابخانه با هم قاطی شوند.

activity_main.xml — root

```xml
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="#F5F7FA">
```

تگ نام ویژگی (attribute) مقدار

این خط را فقط **یک بار و در ریشه (اولین عنصر)** می‌نویسیم — هیچ‌وقت در فرزندان تکرار نمی‌شود.

بخش ۴ • ادامه

ابعاد و جهت‌گیری: چهار ویژگی کلیدی

```
android:layout_width="match_parent"
android:layout_height="match_parent"
```

`layout_width` = عرض | `layout_height` = ارتفاع

`match_parent` = کل فضای موجود را پر کن → تشبیه: فرش را تا آخر اتاق پهن کن

`wrap_content` = به اندازه خود محتوا → تشبیه: بالش به اندازه سرت

جهت‌گیری

```
android:orientation="vertical"
```

`vertical` — زیر هم



`horizontal` — کنار هم



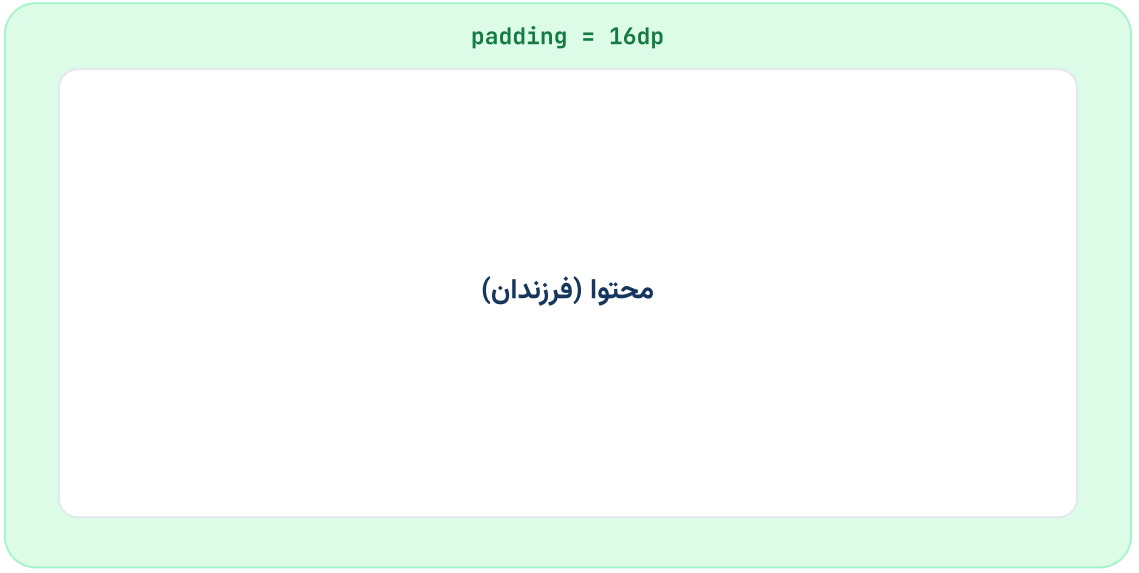
نکته مهم: اگر `horizontal` می‌گذاشتیم، عنوان و دکمه‌ها کنار هم در یک خط می‌رفتند و صفحه خراب می‌شد — پس `vertical` برای صفحه اصلی درست است.

بخش ۴ • ادامه

فاصله داخلی: `padding` و واحد `dp`

```
android:padding="16dp"
```

- `padding` = **فاصله داخلی** — فاصله بین لبه‌های `LinearLayout` و فرزندانش
- `16dp` یعنی ۱۶ دِیپ



dp چیست؟

`dp` = `density-independent pixel` = **پیکسل مستقل از چگالی.**

چرا dp و نه px؟

`px` روی گوشی‌های مختلف اندازه متفاوت دارد (گوشی با صفحه کوچک در برابر بزرگ). `dp` روی همه گوشی‌ها تقریباً یکسان دیده می‌شود.

برای کلاس بگو: همیشه برای فاصله‌ها و اندازه‌ها از `dp` استفاده کن. `px` فقط برای خطوط نازک و مرزها استفاده می‌شود.

بخش ۴ • ادامه

رنگ پس‌زمینه: خواندن کد #F5F7FA

```
android:background="#F5F7FA"
```

رنگ پس‌زمینه = **یک خاکستری خیلی روشن.**

#	F5	F7	FA
این یک کد رنگ است	قرمز (Red)	سبز (Green)	آبی (Blue)

یعنی `RGB = (245, 247, 250)` — **یک خاکستری خیلی روشن و ملایم**

`F5F7FA` `17365D` (رنگ متن عنوان)

برای رنگ شفاف از ۸ رقم استفاده می‌کنیم (مثل `#FFF5F7FA` که `FF` یعنی شفافیت کامل). در این فایل ۶ رقمی است، پس **کاملاً مات** است.

بخش ۵

TextView اول: نمایش عنوان برنامه

TextView چیست؟

یعنی نمایش متن — متنی که فقط خوانده می‌شود، نه نوشته.

مثل تابلو — فقط نگاهش می‌کنی، نمی‌توانی رویش بنویسی

شناسه (id)

`id` = شناسه (اسم). `@+id/` یعنی یک شناسه جدید بساز با این اسم.

در جاوا با همین شناسه عنصر را پیدا می‌کنیم:

```
TextView txt = findViewById(R.id.txtAppTitle);
```

مثل پلاک خانه — هر خانه پلاک دارد تا پستچی پیدا کند

TextView — title

```
<TextView
      android:id="@+id/txtAppTitle"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="رویدادهای دانشگاه"
      android:textSize="25sp"
      android:textStyle="bold"
      android:textColor="#17365D"
      android:gravity="center"
      android:layout_marginTop="8dp"
      android:layout_marginBottom="8dp" />
```

مهم‌ترین خط

★ علامت + یعنی بساز. دفعات بعد بدون + می‌نویسیم: `@id/txtAppTitle` — فقط اولین تعریف + دارد.

بخش ۵ • ادامه

ابعاد و متن: از match_parent تا sp

activity_main.xml

```xml
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:text="رویدادهای دانشگاه"
android:textSize="25sp"
```

عرض `match_parent` = کل عرض صفحه (تا متن وسط چیده شود) — **تشبیه: فرش تا آخر اتاق**

ارتفاع `wrap_content` = به اندازه خود متن، نه بیشتر — **تشبیه: بالش به اندازه سرت**

`text` = متنی که نمایش داده می‌شود

`textSize` = اندازه متن

sp چیست؟

`sp` = `scale-independent pixel` = پیکسل مستقل از مقیاس.

واحد	کاربرد
`dp`	فاصله‌ها، اندازه‌ها، حاشیه‌ها
`sp`	اندازه متن

چرا؟ کاربر می‌تواند در تنظیمات گوشی اندازه فونت را بزرگ‌تر کند. sp این را احترام می‌گذارد، ولی dp نه.

نکته طلایی برای کلاس

برای متن از `sp` استفاده کن، برای بقیه چیزها از `dp` — این یک قاعده مهم است!

استایل متن و نکته مهم gravity

activity_main.xml

```
android:textStyle="bold"
android:textColor="#17365D"
android:gravity="center"
```

B `textStyle bold` = متن ضخیم (پُررنگ) شود. گزینه‌ها:

`normal` = معمولی | `bold` = ضخیم | `italic` = کج

`textColor` `#17365D` = آبی تیره — `RGB = (23, 54, 93)`

`#17365D`

`gravity center` = متن در وسط عنصر قرار بگیرد (افقی و عمودی)

رویدادهای دانشگاه

تفاوت gravity و layout_gravity — نکته مهم!

ویژگی	معنی	مثال
`gravity`	داخل عنصر، فرزندانش کجا باشند	متن داخل TextView وسط باشد
`layout_gravity`	خود عنصر در والدش کجا باشد	TextView در LinearLayout وسط باشد

! `gravity` = چیدمان داخلی. `layout_gravity` = موقعیت خودِ عنصر. **این دو را قاطی نکن!**

بخش ۵ • ادامه

margin یا padding؟ فاصله بیرونی و داخلی

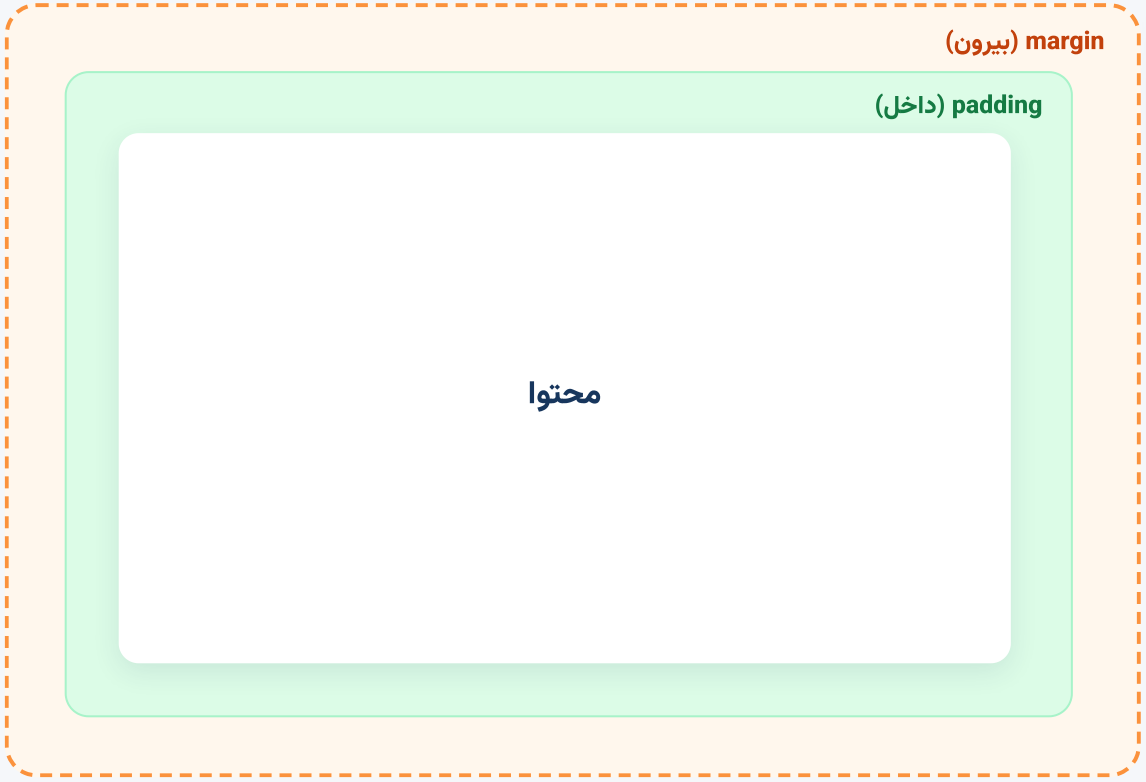
activity_main.xml

```xml
android:layout_marginTop="8dp"
android:layout_marginBottom="8dp"
```

`margin` = فاصله بیرونی — بین این عنصر و بقیه عناصر

`padding` = فاصله داخلی — بین لبه‌های عنصر و محتوایش

	margin	padding
کجا؟	بیرون	داخل
تشبیه	فاصله بین دو خانه	فاصله دیوار تا مبل



`marginTop = 8dp` یعنی دکمه **۸dp** از عنصر بالایی فاصله دارد

برای کلاس بگو: یک مستطیل روی تخته بکش — داخلش `padding` ، بیرونش `margin` . این تفاوت را همه یاد می‌گیرند.

بخش ۶ و ۸

دو TextView ساده: زیرعنوان و عنوان بخش

TextView دوم: توضیح کوتاه

activity_main.xml

```xml
<TextView
    android:id="@+id/txtWelcome"
    ...
    android:text="رویدادهای آموزشی، فرهنگی و دانشجویی"
    android:textSize="14sp"
    android:textColor="#555555"
    android:gravity="center"
    android:layout_marginBottom="12dp" />
```

- اندازه متن کوچک‌تر: `14sp` در برابر `25sp`
- رنگ خاکستری `#555555` (نه آبی تیره)
- بدون `textStyle bold` — معمولی

زیرعنوانی که توضیح می‌دهد برنامه درباره چیست

عنوان بخش فهرست رویدادها (بخش ۸)

activity_main.xml

```xml
<TextView
    ...
    android:text="فهرست رویدادها"
    android:textSize="19sp"
    android:textStyle="bold"
    android:textColor="#17365D"
    android:layout_marginTop="18dp"
    android:layout_marginBottom="8dp" />
```

عنوان بخش است — می‌گوید از اینجا به بعد، فهرست رویدادها شروع می‌شود.

نکته: این TextView اصلاً id ندارد!

چون در جاوا به آن نیازی نداریم — فقط برای نمایش است. هر عنصری که در جاوا به آن نیاز نداریم، `id` نمی‌دهیم. `id` فقط برای عناصری است که می‌خواهیم از جاوا کنترل کنیم.

بخش ۷

سه دکمه اصلی برنامه

Button چیست؟

یعنی دکمه — عنصری قابل کلیک که کاربر می‌تواند رویش بزند.

شناسه	متن دکمه	کارش (در جاوا)
`btnShowEvent`	مشاهده رویداد نمونه	باز کردن جزئیات اولین رویداد
`btnAddEvent`	افزودن رویداد	باز کردن فرم افزودن
`btnSettings`	تنظیمات	فعلاً فقط پیام

سه Button

```
<Button
      android:id="@+id/btnShowEvent"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:text="مشاهده رویداد نمونه" />

<Button
      android:id="@+id/btnAddEvent"
      ...
      android:layout_marginTop="8dp" />

<Button
      android:id="@+id/btnSettings"
      ...
      android:layout_marginTop="8dp" />
```

دکمه دوم و سوم **۸dp** از دکمه بالایی فاصله دارند. دکمه اول فاصله ندارد چون بالاتر از همه است — بعد از `txtWelcome` می‌آید که خودش `marginBottom` دارد.

بخش ۷ • ادامه

قرارداد نام‌گذاری شناسه‌ها

`btn` مخفف `button` است — یک قرارداد نام‌گذاری بین برنامه‌نویس‌ها.

پیشوند	معنی
`btn`	`Button` (دکمه)
`txt`	`TextView` (متن)
`edt`	`EditText` (کادر نوشتن)
`spn`	`Spinner` (لیست کشویی)
`layout`	`LinearLayout` (قوطی چیدمان)

در همین فایل:

`txtAppTitle` `txtWelcome` `btnShowEvent` `btnAddEvent`

`btnSettings` `layoutEventList`

متن دکمه قوطی چیدمان

نکته برای کلاس

این پیشوندها اجباری نیستند، ولی همه برنامه‌نویس‌ها از آن‌ها استفاده می‌کنند. با یک نگاه به کد می‌فهمی این عنصر دکمه است یا متن.

بخش ۹ – مهم‌ترین بخش

ScrollView: قوطی اسکرول‌شونده

ScrollView چیست؟

قوطی اسکرول‌شونده – اگر محتوایش بیشتر از فضای صفحه باشد، کاربر می‌تواند اسکرول کند (بالا-پایین بکشد).

چرا لازم است؟

فهرست رویدادها می‌تواند خیلی بلند شود (مثلاً ۵۰ رویداد). بدون `ScrollView` ، رویدادهای پایینی از صفحه بیرون می‌زدند و دیده نمی‌شدند.

در جاوا چه می‌شود؟

```
layoutEventList = findViewById(R.id.layoutEventList);
layoutEventList.addView(itemLayout, params);
```

یعنی هر رویداد را به این قوطی اضافه کن.

ScrollView + container

```
<ScrollView
      android:layout_width="match_parent"
      android:layout_height="0dp"
      android:layout_weight="1">

      <LinearLayout
          android:id="@+id/layoutEventList"
          android:layout_width="match_parent"
          android:layout_height="wrap_content"
          android:orientation="vertical" />

</ScrollView>
```

ترفند اسلاید بعد!

`ScrollView` **فقط یک فرزند مستقیم می‌پذیرد!** برای همین یک `LinearLayout` گذاشتیم که همه رویدادها را نگه دارد. چند چیز اسکرول‌شونده می‌خواهی؟ همه را داخل یک LinearLayout بگذار و آن را داخل ScrollView.

بخش ۹ • نکته کلیدی

ترفند کلیدی: `0dp + weight = 1`

- `layout_weight` یعنی: وزن این عنصر در تقسیم فضای باقی‌مانده.
- چرا `0dp` ؟ یعنی ارتفاع من را از فضای باقی‌مانده حساب کن، نه از محتوایم.

activity_main.xml

```
android:layout_height="0dp"
android:layout_weight="1"
```

تشبیه: تقسیم کیک

فرض کن یک کیک داری و چند نفر. `weight` یعنی سهم هر نفر چقدر باشد. اگر یک نفر `weight=1` باشد و بقیه `0` ، کل کیک را می‌گیرد.

۱ `LinearLayout` اصلی همه فرزندانش را می‌چیند

۲ عنوان‌ها و دکمه‌ها `wrap_content` هستند → فضای لازم خودشان را می‌گیرند

۳ `ScrollView` با `weight=1` → بقیه فضای صفحه را می‌گیرد

نتیجه: فهرست رویدادها همیشه پرکننده فضای باقی‌مانده است — اگر بلند شد، اسکرول می‌شود.

ترکیب `0dp + weight=1` یک الگوی کلاسیک در اندروید است: بقیه فضا را پر کن. حتماً به خاطر بسپارید!

بخش ۱۰

ارتباط XML با MainActivity.java

در XML	در MainActivity.java
`@+id/txtAppTitle`	*(استفاده نشده — فقط نمایش)*
`@+id/txtWelcome`	*(استفاده نشده — فقط نمایش)*
`@+id/btnShowEvent`	`findViewById(R.id.btnShowEvent)`
`@+id/btnAddEvent`	`findViewById(R.id.btnAddEvent)`
`@+id/btnSettings`	`findViewById(R.id.btnSettings)`
`@+id/layoutEventList`	`findViewById(R.id.layoutEventList)`

پیدا و کنترل در جاوا — استفاده نشده — فقط نمایش

> در XML می‌سازیم و نام‌گذاری می‌کنیم. در جاوا با findViewById پیدا و کنترل می‌کنیم — این پل ارتباطی دو فایل است.

R چیست؟

R یک کلاس تولیدشده خودکار است. وقتی XML می‌نویسی، اندروید خودش کلاس R را می‌سازد که همه منابع (layout ، id ، string ، color و...) را در خود دارد.

مثل فهرست مطالب کتاب — اندروید فهرست را می‌سازد تا تو راحت پیدایش کنی

`R.layout.activity_main` ← فایل `activity_main.xml`

`R.id.btnAddEvent` ← شناسه `btnAddEvent`

نکته خطایابی

اگر R را قرمز دیدی (خطا)، یعنی یک مشکل در XML داری (مثلاً تگ بسته نشده). اول XML را درست کن — R خودش درست می‌شود.

بخش ۱۱

چرا این چیدمان این‌طور است؟ — نگاه از بالا به پایین

۱ **بالا: عنوان و توضیح**

کاربر می‌فهمد کجاست

۲ **وسط: دکمه‌های عملیات**

کاربر چه کاری می‌تواند بکند

۳ **پایین: فهرست**

کاربر نتیجه را می‌بیند

این یک طراحی `mobile-first` است: مهم‌ترین چیزها بالا هستند، چون کاربر اول آنها را می‌بیند.



بخش ۱۲

جمع‌بندی: این فایل را در ۲ دقیقه توضیح بده

۸ نکته‌ای که باید در ذهن بماند

۱

این فایل طراحی ظاهر صفحه اصلی است — با `setContentView` به `MainActivity` وصل می‌شود.

۲

ریشه یک `LinearLayout` عمودی است — همه چیز زیر هم چیده می‌شود.

۳

ترتیب از بالا: عنوان، توضیح، سه دکمه، عنوان بخش، و `ScrollView` فهرست.

۴

هر عنصر مهم یک `id` دارد تا در جاوا با `findViewById` پیدا شود.

۵

`match_parent` = کل فضا | `wrap_content` = به اندازه محتوا.

۶

`dp` برای فاصله‌ها | `sp` برای اندازه متن.

۷

`margin` = فاصله بیرونی | `padding` = فاصله داخلی.

۸

`ScrollView` با `0dp + weight=1` بقیه فضا را می‌گیرد — اگر فهرست بلند شد، اسکرول می‌شود.

بخش ۱۳ • پرسش و پاسخ

پرسش و پاسخ کلاس (۱ از ۳)

س: `match_parent` و `wrap_content` چه فرقی دارند؟

ج: `match_parent` = کل فضای موجود را پر کن. `wrap_content` = فقط به اندازه محتوا.

س: `dp` و `sp` چه فرقی دارند؟

ج: `dp` برای فاصله‌ها و اندازه‌ها. `sp` برای اندازه متن — چون به تنظیمات فونت کاربر احترام می‌گذارد.

س: `margin` و `padding` چه فرقی دارند؟

ج: `margin` = فاصله بیرونی (بین عناصر). `padding` = فاصله داخلی (بین لبه و محتوا).

بخش ۱۳ • پرسش و پاسخ

پرسش و پاسخ کلاس (۲ از ۳)

س: `gravity` و `layout_gravity` چه فرقی دارند؟

ج: `gravity` = چیدمان داخلی (فرزندان). `layout_gravity` = موقعیت خودِ عنصر در والد.

س: چرا `ScrollView` فقط یک فرزند دارد؟

ج: چون `ScrollView` فقط یک فرزند مستقیم می‌پذیرد. برای همین یک `LinearLayout` گذاشتیم که همه چیز را در خودش نگه دارد.

س: `0dp + weight=1` چه معنایی دارد؟

ج: ارتفاع من را از فضای باقی‌مانده حساب کن، نه از محتوایم — یعنی بقیه فضا را پر کن.

پرسش و پاسخ کلاس (۳ از ۳)

`@+id/` و `@id/` چه فرقی دارند؟

`@+id/` = شناسه جدید بساز. `@id/` = به شناسه موجود اشاره کن. علامت `+` فقط در اولین تعریف می‌آید.

`xmlns:android` چیست؟

فضای نام `XML` — می‌گوید همه چیزهایی که با `android:` شروع می‌شوند از کتابخانه اندروید هستند. فقط یک بار در ریشه می‌آید.

چرا بعضی عناصر id ندارند؟

چون در جاوا به آنها نیاز نداریم — `id` فقط برای عناصری است که می‌خواهیم از جاوا کنترل کنیم.

چرا رنگ `#F5F7FA` است؟

یک خاکستری خیلی روشن و ملایم — `F5` قرمز، `F7` سبز، `FA` آبی. `RGB = (245, 247, 250)` .

بخش ۱۴ • پایان

مقایسه نهایی فایل‌ها — برای درک کلی

فایل	زبان	نقش
`activity_main.xml`	XML	طراحی صفحه اصلی
`activity_event_detail.xml`	XML	طراحی صفحه جزئیات
`dialog_add_event.xml`	XML	طراحی فرم افزودن
`MainActivity.java`	Java	منطق صفحه اصلی
`EventDetailActivity.java`	Java	منطق صفحه جزئیات
`Event.java`	Java	مدل داده

`XML` ظاهر را می‌سازد، `Java` منطق را. با `id` به هم وصل می‌شوند. مثل این است که XML خانه را بسازد و Java برق و آب را وصل کند.

آماده پرسش‌های شما هستم — موفق باشید!